INSTITUTO TECNOLÓGICO DE SONORA

Distribuidora de perecederos

Avance 01 - El dominio y la arquitectura

Versión corregida a partir de la revisión de la primera entrega.

Materia: Frontend Web: Temas Emergentes

Profesor: Carlos Alberto Ruiz Castrejón

Equipo 4
Daniel Alejandro Arce Galván - 00000262142
Jesus Armando Arce Baldenebro - 00000206661
Ana Isabel González Mendivil - 00000244319
Javi - 00000164512

Entrega original: 29 de septiembre de 2026
Revisión: 19 de octubre de 2026

Repositorio: https://github.com/Arc-Dan/Proyecto-Distribuidora-Carne

1. Objetivo y vocabulario del dominio

Administrar la operación de una distribuidora de carnes que abastece a restaurantes, reduciendo las pérdidas por caducidad mediante el control del inventario por lotes y la rotación de mercancía según su vencimiento.

La elección responde a una necesidad práctica de pequeños comercios locales. El proyecto permite aplicar el control de existencias, la trazabilidad y la separación de reglas de negocio en una situación cercana al entorno del equipo.

Glosario

Producto: Tipo de carne comercializado, con presentación, unidad de medida, precio base y stock mínimo.

Lote: Entrada de un producto con proveedor, cantidad disponible y fechas de ingreso y caducidad propias.

Pedido: Solicitud de un restaurante. Su estado permite distinguir recibido, surtido, en ruta y entregado.

Cliente (restaurante): Restaurante que solicita productos. En el código se referencia mediante clienteId; los usuarios con rol cliente representan este actor.

Merma: Pérdida real de mercancía por caducidad, daño o faltante. Descuenta existencias y conserva el motivo y la fecha.

Stock mínimo: Reserva de producto vigente que debe quedar después de surtir un pedido. No bloquea el registro de pérdidas reales.

Proveedor: Origen de la mercancía de un lote. En este avance se registra como texto, sin entidad propia.

Surtido: Asignación y descuento de lotes para atender un pedido completo. Genera movimientos que permiten rastrear lo entregado.

FEFO: First Expired, First Out: primero se utiliza el lote vigente cuya caducidad sea más próxima.

Existencia disponible: Suma de cantidades positivas de lotes ya ingresados y aún vigentes de un producto.

Movimiento de surtido: Registro que relaciona un pedido, un producto y las cantidades tomadas de cada lote.

2. Reglas de negocio

RN01. No puede surtirse un lote cuya fecha de caducidad sea igual o anterior al instante de la operación, porque el producto ya no es apto para la venta. Los lotes caducados se excluyen de la existencia disponible.

RN02. No puede surtirse una cantidad que deje la existencia vigente por debajo del stock mínimo, porque se comprometería la reserva operativa. Sí se permite dejar exactamente el mínimo.

RN03. No puede surtirse nuevamente un pedido que ya esté surtido, en ruta o entregado, porque se duplicarían la salida de inventario y los movimientos del mismo pedido.

RN04. No puede surtirse una cantidad superior a la existencia vigente disponible, porque no se puede comprometer mercancía inexistente o caducada.

RN05. No puede surtirse un pedido vacío ni registrar cantidades cero, negativas o no finitas, porque no representan una cantidad operativa válida. Las cantidades de líneas repetidas se suman antes de validar el producto.

RN06. No puede registrarse una merma mayor que la cantidad disponible del lote, porque produciría una existencia negativa. Una merma real sí puede dejar el inventario por debajo del mínimo.

RN07. No puede operarse sobre un producto, lote o pedido inexistente, porque se perdería la trazabilidad de las operaciones.

RN08. No puede ingresarse un lote con fechas inválidas, con caducidad anterior o igual al ingreso, o ya caducado, porque no representa mercancía vigente. Un lote con ingreso futuro no participa en el surtido actual.

RN09. No puede elegirse un lote de vencimiento posterior mientras exista uno vigente de vencimiento más próximo, porque se aumentaría el riesgo de merma. Se aplica FEFO y, en empate, el menor identificador.

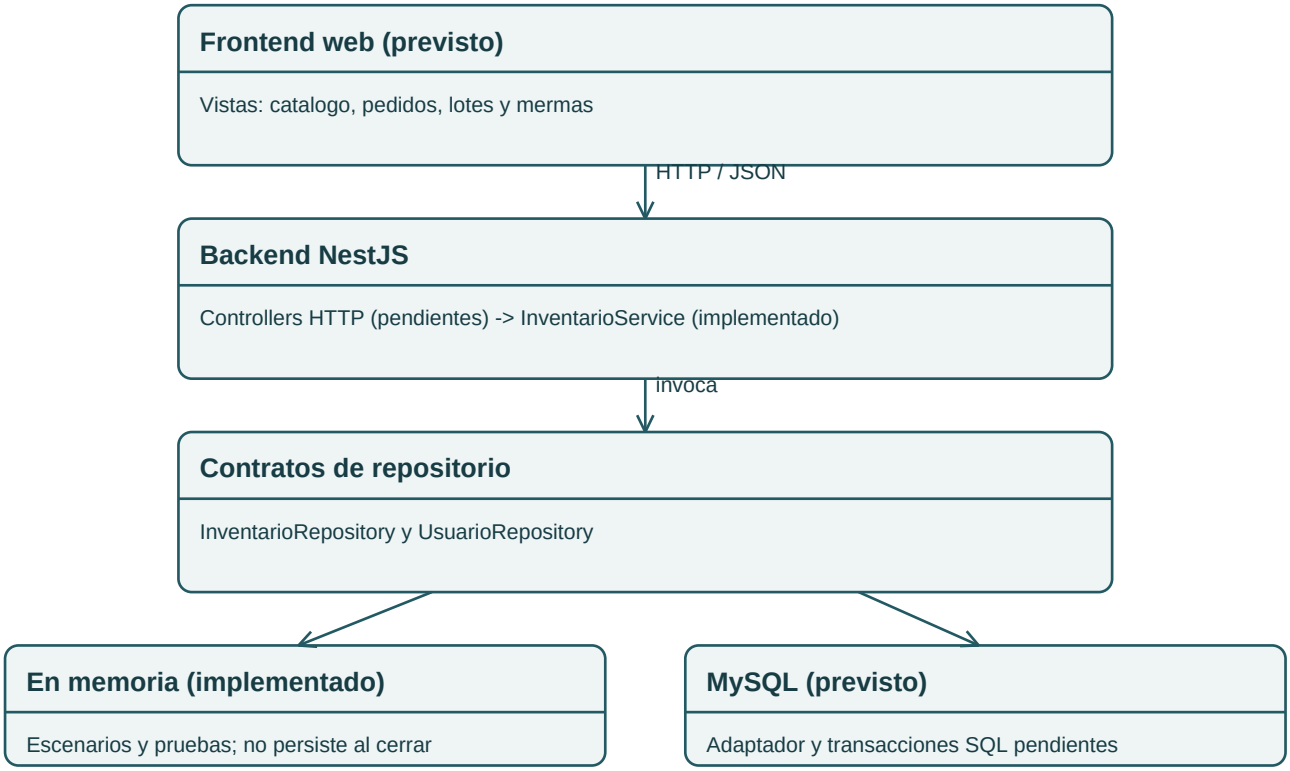
RN10. No puede confirmarse parcialmente un surtido de varios productos, porque el pedido quedaría inconsistente. Todos los descuentos, movimientos y el cambio de estado se confirman juntos o se revierten.

Criterio temporal

Las fechas se comparan como instantes Date. La caducidad es un límite exclusivo: fechaCaducidad > ahora. Si el negocio captura solo fechas de calendario, la capa de entrada deberá acordar su conversión al instante de vencimiento y la zona horaria antes de habilitar la interfaz.

3. Arquitectura y contrato entre capas

La arquitectura propuesta es cliente-servidor. La vista permite consultar y capturar información; el backend aplica las reglas del negocio; los repositorios aíslan el dominio del mecanismo de almacenamiento. MySQL sigue siendo la persistencia prevista por sus relaciones y soporte transaccional.



Estado real de este avance

Se implementaron InventarioService, InMemoryInventarioRepository e InMemoryUsuarioRepository. El dominio se ejecuta directamente con escenarios, sin depender de HTTP ni de una base de datos. El frontend, los endpoints del negocio y el adaptador MySQL corresponden a trabajo posterior.

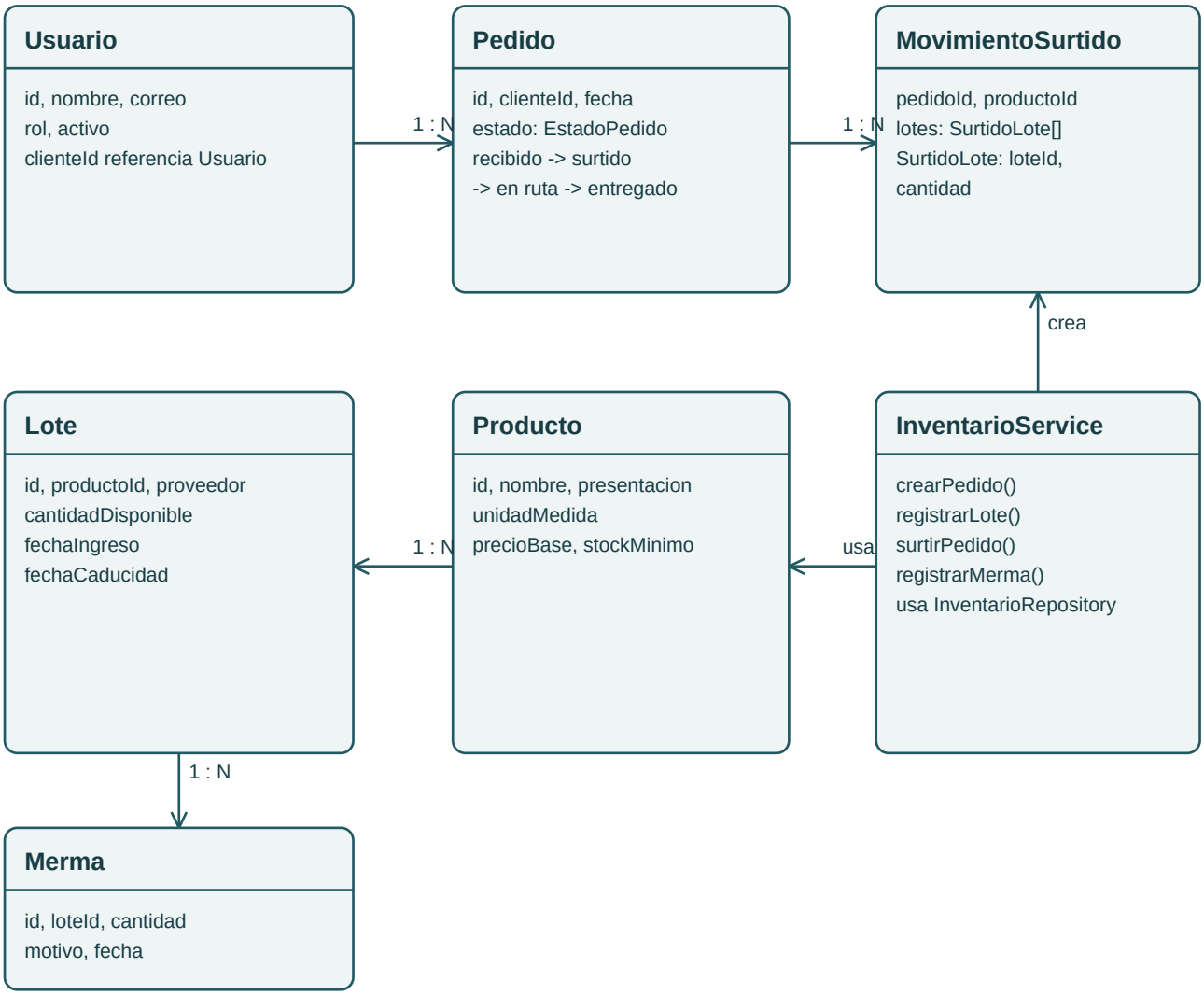
Contrato propuesto

La vista enviará identificadores y datos de captura. Para surtir se recibe un pedidoId y una lista de líneas {productoId, cantidad}; para merma, {loteId, cantidad, motivo}. El servicio devuelve movimientos de surtido o la merma creada; ante una restricción lanza un error de dominio específico. La traducción a respuestas HTTP corresponde a los futuros controllers.

El repositorio expone operaciones asíncronas y enTransaccion(). La implementación en memoria serializa las operaciones del servicio y restaura el estado ante errores; el futuro adaptador MySQL deberá cumplir el mismo contrato mediante transacciones y bloqueo apropiado. Los datos en memoria se pierden al finalizar el proceso.

4. Componentes y relaciones

Diagrama de clases y contratos principales. Las interfaces TypeScript representan entidades; el servicio concentra el comportamiento y depende del contrato del repositorio.



Tipos y alcance

EstadoPedido: recibido, surtido, en ruta, entregado. MotivoMerma: caducidad, daño, faltante. NuevoLote se define como Omit<Lote, 'id'>: conserva producto, proveedor, cantidad y ambas fechas.

Las líneas se entregan al caso de uso de surtido; este avance no guarda un detalle solicitado antes de surtir. Los movimientos sí conservan las cantidades efectivamente tomadas. La validación de que clienteId corresponda a un restaurante activo y las transiciones de reparto quedan pendientes de la siguiente entrega.

Los repositorios devuelven copias de las entidades para evitar cambios accidentales desde fuera del almacenamiento. El repositorio de usuarios implementa listado, consulta, creación, actualización y eliminación; la creación usa rol cliente y activo verdadero como valores por defecto.

5. Escenarios reproducibles y correcciones

Desde distribuidora-app ejecutar npm ci y luego npm run escenarios. El programa termina con código distinto de cero ante una aserción fallida. No requiere MySQL.

Escenario 1. Surtido válido y duplicado

Con 20 kg vigentes y 50 kg caducados, se solicitan 12 kg. Se toman 10 kg del lote que vence primero y 2 kg del siguiente. El lote caducado no cambia; el pedido pasa a surtido y un segundo intento se rechaza sin otro movimiento.

Escenario 2. Restricciones de inventario

Solicitar 21 kg vigentes se rechaza por insuficiencia; solicitar 16 kg se rechaza porque dejaría menos de la reserva de 5 kg. Un pedido de varios productos cuyo segundo producto no tenga existencias se revierte completo.

Escenario 3. Mermas

Se rechazan cantidades cero, negativas, NaN, infinitas y mayores a la existencia. Una merma por los 50 kg del lote caducado descuenta su saldo a cero y registra una sola pérdida.

Verificación adicional

npm test -- --runInBand ejecuta los escenarios y pruebas de límites del stock mínimo, referencias inexistentes, fechas, aislamiento de datos, contrato de usuarios y surtidos concurrentes. npm run build comprueba la compilación. npm run test:e2e valida el endpoint inicial existente de NestJS.

Correcciones atendidas

Glosario ampliado; reglas restrictivas documentadas; ambos diagramas incorporados en este PDF; servicio de inventario implementado; repositorios en memoria operables; tres escenarios con aserciones; tipo NuevoLote corregido.